

ÉDITION 2026

GUIDE COMPLET

CRÉATION D'UN BOT TELEGRAM

Apprendre à coder, publier et faire vivre
son propre bot, pas à pas, avec Python

```
>>> application.run_polling()
```

UN OUVRAGE SIGNÉ

Le H

À PROPOS DE CE GUIDE

Bienvenue dans votre guide

Ce guide vous accompagne, du tout début à la mise en production, dans la création d'un bot Telegram fonctionnel avec Python. Il s'adresse à des personnes qui n'ont jamais écrit une ligne de code : chaque notion technique est expliquée avant d'être utilisée, et chaque exemple est complet et testable.

L'objectif n'est pas seulement de comprendre la théorie, mais de terminer ce guide avec un bot réellement en ligne, capable de répondre à des messages, de proposer des menus, de retenir des informations et de tourner en continu sur un serveur.

COMMENT LIRE CE GUIDE

Les modules 1 à 6 posent les bases indispensables et doivent être suivis dans l'ordre. À partir du module 7, chaque module ajoute une fonctionnalité indépendante que vous pouvez consulter selon vos besoins.

Ce que vous allez construire

Tout au long de ce guide, vous ferez évoluer un seul et même projet, petit à petit, jusqu'à obtenir un bot complet capable de :

- Répondre à des commandes et à des messages texte
- Afficher des menus et des boutons cliquables
- Mener une conversation en plusieurs étapes (formulaire, quiz, commande guidée)
- Retenir des informations d'un utilisateur à l'autre
- Envoyer et recevoir des images et des fichiers
- Continuer à fonctionner même en cas d'erreur imprévue
- Tourner 24h/24 sur un serveur, sans rester allumé sur votre ordinateur

Table des matières

- 01 Comprendre les bots Telegram** — Concepts, fonctionnement et cas d'usage
- 02 Préparer son environnement** — Python, éditeur de code et terminal
- 03 Les bases de Python essentielles** — Le minimum pour suivre ce guide sereinement
- 04 Créer son bot avec BotFather** — Obtenir un nom, un identifiant et un token
- 05 Installer python-telegram-bot** — Environnement virtuel et structure de projet
- 06 Son premier bot fonctionnel** — Un bot qui répond à un message
- 07 Comprendre les handlers** — Commandes, messages et filtres
- 08 Créer des commandes personnalisées** — Construire une vraie interface conversationnelle
- 09 Boutons et menus interactifs** — Claviers en ligne et claviers personnalisés
- 10 Gérer une conversation à états** — Le ConversationHandler en détail
- 11 Stocker des données** — Fichiers JSON et base de données SQLite
- 12 Envoyer et recevoir des médias** — Photos, documents et fichiers
- 13 Gérer les erreurs et journaliser** — Rendre son bot robuste et observable
- 14 Sécuriser son bot** — Variables d'environnement et bonnes pratiques
- 15 Héberger et déployer son bot** — Faire tourner son bot 24h/24
- 16 Aller plus loin** — Tâches planifiées, API externes et IA
- 17 Votre feuille de route** — Passer de la théorie à un bot en production
- Conclusion & glossaire** — Synthèse, lexique et ressources pour aller plus loin

01

MODULE 01

Comprendre les bots Telegram

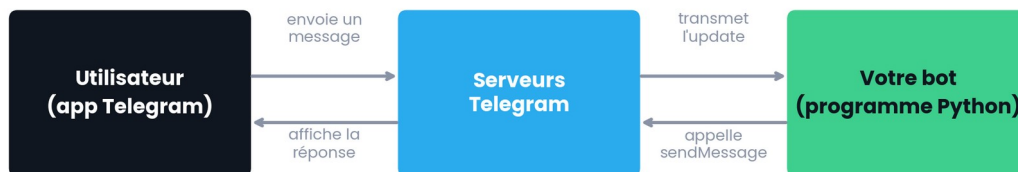
Concepts, fonctionnement et cas d'usage



Comprendre les bots Telegram

Un bot Telegram est un compte spécial sur Telegram, piloté non pas par une personne mais par un programme informatique. Du point de vue de l'utilisateur, discuter avec un bot ressemble exactement à discuter avec un contact classique : on lui envoie des messages, il répond. La différence se joue entièrement côté coulisses : chaque message reçu déclenche l'exécution de votre code, qui décide de la réponse à renvoyer.

Comment circule un message dans un bot Telegram



Telegram sert d'intermédiaire entre l'utilisateur et votre programme.

Ce mécanisme repose sur un principe simple : Telegram met à disposition une interface de programmation (une API) qui permet à n'importe quel programme de lire les messages reçus par un compte bot et d'y répondre. Ce guide vous apprend à écrire ce programme en Python.

Pourquoi créer un bot Telegram ?

Les bots Telegram sont utilisés pour des besoins très variés, du simple gadget personnel à l'outil professionnel. Voici quelques exemples représentatifs des cas d'usage les plus courants.

Cas d'usage	Exemple concret
Notifications automatiques	Recevoir une alerte quand un prix baisse ou qu'un événement se produit
Assistant personnel	Prendre des notes, gérer une liste de tâches, répondre à des questions

Cas d'usage	Exemple concret
Support client	Répondre aux questions fréquentes d'une communauté ou d'une clientèle
Modération de groupe	Filtrer les messages indésirables, accueillir les nouveaux membres
Interface pour un service	Commander un produit, réserver un créneau, consulter des données
Jeux et divertissement	Quiz, jeux textuels, générateurs de contenu

Pourquoi Telegram plutôt qu'une autre plateforme ?

Telegram s'est imposé comme la plateforme de référence pour créer des bots, pour plusieurs raisons concrètes qui facilitent la vie d'un développeur débutant :

- L'API est gratuite, bien documentée et ne nécessite aucune validation préalable
- La création d'un bot prend moins de deux minutes, directement depuis l'application
- Aucune carte bancaire ni compte développeur payant n'est requis pour démarrer
- La communauté est large : la quasi-totalité des problèmes rencontrés ont déjà une réponse en ligne
- Les bots peuvent être utilisés en message privé, dans des groupes et dans des canaux

À RETENIR

Un bot Telegram n'est jamais « allumé » par magie : il doit être exécuté par un programme qui tourne quelque part (votre ordinateur pendant les tests, puis un serveur pour un usage réel). C'est ce programme que vous allez apprendre à écrire dans ce guide.

Ce que peut faire un bot... et ce qu'il ne peut pas faire

Avant de se lancer, il est utile de connaître les limites du système, pour éviter de construire un projet impossible à réaliser.

Un bot peut	Un bot ne peut pas
Recevoir et envoyer des messages, images, fichiers	Démarrer une conversation avec un utilisateur qui ne l'a jamais contacté
Afficher des boutons et des menus interactifs	Accéder aux messages échangés entre deux humains sans lui

Un bot peut	Un bot ne peut pas
Être ajouté à un groupe ou un canal	Lire l'historique des messages envoyés avant son ajout à un groupe
Répondre instantanément, 24h/24 si hébergé correctement	Fonctionner sans qu'un programme tourne quelque part

Le vocabulaire de base à connaître

Quelques termes reviendront tout au long de ce guide. Les définir dès maintenant évite toute confusion par la suite.

Terme	Définition
Token	La clé secrète qui identifie votre bot et vous permet de le contrôler
API Telegram	L'ensemble des fonctions que Telegram met à disposition pour piloter un bot
Update	Un événement reçu par le bot (nouveau message, clic sur un bouton, etc.)
Handler	Une fonction de votre code chargée de réagir à un type d'update précis
Polling	Méthode où le bot interroge Telegram en boucle pour savoir s'il y a du nouveau
Webhook	Méthode où Telegram prévient directement votre serveur à chaque update

02

MODULE 02

Préparer son environnement

Python, éditeur de code et terminal



Préparer son environnement

Avant d'écrire la moindre ligne de code, il faut installer les outils qui permettent d'écrire, d'exécuter et de tester un programme Python. Cette étape ne prend que quelques minutes et n'a besoin d'être faite qu'une seule fois.

Installer Python

Python est le langage de programmation utilisé dans ce guide. Il est gratuit et disponible sur Windows, macOS et Linux.

1. Rendez-vous sur **python.org/downloads** depuis votre navigateur.
2. Téléchargez la dernière version stable proposée pour votre système d'exploitation.
3. **Lancez l'installateur.** Sur Windows, cochez impérativement la case « Add Python to PATH » avant de cliquer sur Installer.
4. Une fois l'installation terminée, ouvrez un terminal pour vérifier que tout fonctionne (voir ci-dessous).

Vérifier l'installation

Un terminal est une fenêtre qui permet de taper des commandes textuelles plutôt que de cliquer sur des icônes. Sur Windows, cherchez « Invite de commandes » ou « PowerShell » ; sur macOS, cherchez « Terminal » ; sur Linux, il est généralement accessible directement.

```
$ python --version
```

Cette commande doit afficher un numéro de version, par exemple « Python 3.12.4 ». Si une erreur apparaît, relancez l'installateur Python en cochant bien l'option d'ajout au PATH.

Installer un éditeur de code

Un éditeur de code est un logiciel conçu pour écrire du code facilement (coloration syntaxique, suggestions, détection d'erreurs). Visual Studio Code (VS Code), gratuit et disponible sur toutes les plateformes, est recommandé pour ce guide.

1. Téléchargez VS Code depuis **code.visualstudio.com**.
2. Installez-le en conservant les options par défaut.
3. Une fois ouvert, installez l'extension « Python » (icône de puzzle sur le côté gauche, puis recherche « Python », puis Installer).

BON À SAVOIR

VS Code intègre un terminal directement dans son interface (menu Terminal > Nouveau terminal). Vous n'aurez donc pas besoin de jongler entre plusieurs fenêtres pour écrire votre code et exécuter vos commandes.

Créer un dossier de projet

Toutes les créations de ce guide vivront dans un même dossier, que nous appellerons « mon-bot-telegram ». Créez-le où vous le souhaitez sur votre ordinateur (par exemple dans vos Documents), puis ouvrez-le dans VS Code via Fichier > Ouvrir un dossier.

Les bases du terminal à connaître

Quelques commandes reviendront régulièrement tout au long de ce guide. Il n'est pas nécessaire de toutes les mémoriser dès maintenant : vous pourrez revenir à ce tableau à tout moment.

Commande	Ce qu'elle fait
cd nom-du-dossier	Se déplacer dans un dossier
dir (Windows) / ls (macOS, Linux)	Lister le contenu du dossier actuel
python nom_du_fichier.py	Exécuter un script Python
Ctrl + C	Arrêter un programme en cours d'exécution
pip install nom_du_paquet	Installer une bibliothèque Python

PIÈGE FRÉQUENT

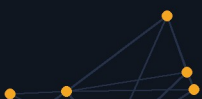
Une erreur « command not found » ou « n'est pas reconnu » signifie presque toujours que Python n'a pas été correctement ajouté au PATH lors de l'installation. Réinstallez Python en cochant bien la case correspondante, ou recherchez « ajouter Python au PATH » suivi du nom de votre système d'exploitation.

03

MODULE 03

Les bases de Python essentielles

Le minimum pour suivre ce guide sereinement



Les bases de Python essentielles

Ce module condense les quelques notions de programmation indispensables pour suivre la suite du guide. Il ne remplace pas un cours complet de Python, mais donne tout le vocabulaire et les mécanismes utilisés dans les exemples de code à venir.

Variables : stocker une information

Une variable est un nom auquel on associe une valeur, pour pouvoir la réutiliser plus loin dans le programme.

```
variables.py  
nom = "Alex"  
age = 28  
est_actif = True
```

Ici, `nom` contient une chaîne de caractères (du texte, entre guillemets), `age` contient un nombre entier, et `est_actif` contient un booléen (Vrai ou Faux).

Fonctions : regrouper des instructions

Une fonction est un bloc de code auquel on donne un nom, pour pouvoir l'exécuter à tout moment sans le réécrire. Elle peut recevoir des informations en entrée (des paramètres) et en renvoyer une en sortie (un retour).

```
fonctions.py  
def saluer(nom):  
    return "Bonjour " + nom  
  
message = saluer("Alex")  
print(message) # Affiche : Bonjour Alex
```

À PROPOS DE L'INDENTATION

En Python, les espaces en début de ligne (l'indentation) ne sont pas décoratifs : ils indiquent quelles lignes appartiennent à un bloc (une fonction, une condition, une boucle). Une indentation incohérente provoque une erreur. La convention est d'utiliser 4 espaces.

Listes et dictionnaires : organiser des données

Une liste rassemble plusieurs valeurs dans un ordre précis. Un dictionnaire associe des clés à des valeurs, un peu comme une fiche avec des champs nommés.

```
collections.py
fruits = ["pomme", "banane", "kiwi"]
print(fruits[0]) # Affiche : pomme

utilisateur = {
    "nom": "Alex",
    "age": 28
}
print(utilisateur["nom"]) # Affiche : Alex
```

Conditions : faire des choix

Une condition permet d'exécuter un bloc de code seulement si une situation précise se présente.

```
conditions.py
age = 15

if age >= 18:
    print("Accès autorisé")
else:
    print("Accès refusé")
```

async et await : exécuter du code qui attend

Cette notion est spécifique mais essentielle pour la suite : envoyer un message via Telegram prend un peu de temps (le temps que la requête voyage sur internet). Pendant ce court instant, un programme classique resterait bloqué, incapable de faire autre chose. Python propose un mécanisme, appelé asynchrone, qui permet au programme de s'occuper d'autre chose pendant l'attente.

```
asynchrone.py
import asyncio

async def dire_bonjour():
    print("Bonjour...")
    await asyncio.sleep(1) # attend 1 seconde sans tout bloquer
    print("...et bonsoir !")

asyncio.run(dire_bonjour())
```

Retenez simplement ceci : une fonction précédée du mot-clé `async` doit être appelée avec `await` lorsqu'on l'exécute depuis une autre fonction `async`. La bibliothèque que nous utiliserons pour Telegram impose cette écriture ; vous la retrouverez dans presque tous les exemples de ce guide, sans avoir besoin d'en maîtriser tous les rouages.

VOUS N'AVEZ PAS BESOIN DE TOUT MAÎTRISER

Ce module donne juste assez de repères pour comprendre le code des modules suivants. Si une notion reste floue, continuez : la pratique sur un vrai projet clarifie presque toujours ce que la théorie seule laisse abstrait.

04

MODULE 04

Créer son bot avec BotFather

Obtenir un nom, un identifiant et un token



Créer son bot avec BotFather

BotFather est lui-même un bot Telegram, créé et maintenu par Telegram, qui sert à créer et gérer tous les autres bots de la plateforme. C'est le point de passage obligé pour obtenir votre propre bot.

Créer le bot, étape par étape

1. **Ouvrez Telegram** (sur votre téléphone ou sur ordinateur) et recherchez « BotFather » dans la barre de recherche.
2. **Vérifiez le compte** avant de continuer : le vrai BotFather possède une coche de vérification bleue.
3. **Démarrez la conversation** en appuyant sur Démarrer ou en envoyant /start.
4. **Lancez la création** en envoyant la commande /newbot.
5. **Choisissez un nom d'affichage** pour votre bot (par exemple « Mon Assistant »). Ce nom est visible par tous et peut être modifié plus tard.
6. **Choisissez un identifiant unique** se terminant obligatoirement par « bot » (par exemple mon_assistant_bot). Cet identifiant ne peut plus être changé et doit être disponible.

Si l'identifiant choisi est déjà pris, BotFather vous le signale et vous invite à en essayer un autre. Une fois la création validée, BotFather vous répond avec un message contenant votre token.

LE TOKEN, VOTRE CLÉ SECRÈTE

Le token ressemble à une longue chaîne du type 7451236890:AAHk3f... Il permet à quiconque le possède de contrôler entièrement votre bot. Ne le partagez jamais publiquement, ne le publiez pas sur un dépôt de code visible par d'autres, et ne l'envoyez pas par un canal non sécurisé.

Sauvegarder son token en sécurité

Pour l'instant, copiez ce token dans un fichier texte temporaire sur votre ordinateur — nous verrons au module 14 la bonne pratique consistant à le stocker dans un fichier séparé et protégé, plutôt que directement dans le code.

Personnaliser son bot (facultatif mais recommandé)

Depuis la conversation avec BotFather, la commande /mybots permet de retrouver votre bot et d'accéder à plusieurs réglages utiles.

Réglage	Ce qu'il permet
/setdescription	Définir le texte affiché avant que l'utilisateur démarre le bot
/setabouttext	Définir le texte affiché sur le profil du bot
/setuserpic	Ajouter une photo de profil au bot
/setcommands	Déclarer la liste des commandes, avec leur description, affichée dans le menu

Retrouver son bot

Une fois créé, votre bot est accessible via un lien du type t.me/votre_identifiant_bot. Ouvrez ce lien et démarrez une conversation : pour l'instant, le bot ne répondra rien, car aucun programme ne tourne encore pour le piloter. C'est justement l'objet des modules suivants.

CHECKLIST AVANT DE CONTINUER

Assurez-vous d'avoir : un bot créé via BotFather, un token copié quelque part, et le lien [t.me](https://t.me/votre_bot_ouvert) de votre bot ouvert dans Telegram pour pouvoir tester facilement.

05

MODULE 05

Installer python-telegram-bot

Environnement virtuel et structure de projet

Installer python-telegram-bot

python-telegram-bot est une bibliothèque : un ensemble de code déjà écrit par d'autres développeurs, que l'on peut réutiliser dans son propre projet. Elle prend en charge toute la communication technique avec les serveurs Telegram, pour que vous puissiez vous concentrer sur le comportement de votre bot plutôt que sur les détails de bas niveau.

Créer un environnement virtuel

Un environnement virtuel est un espace isolé propre à un projet, dans lequel on installe les bibliothèques dont ce projet a besoin, sans interférer avec les autres projets présents sur le même ordinateur. C'est une pratique standard, à adopter dès le premier projet.

Ouvrez un terminal dans votre dossier de projet, puis exécutez :

```
$ python -m venv venv
```

Cette commande crée un dossier « venv » contenant un environnement Python isolé.

Il faut ensuite activer cet environnement, ce qui indique au terminal d'utiliser cette version isolée de Python plutôt que celle installée globalement sur l'ordinateur :

Système	Commande d'activation
Windows (PowerShell)	venv\Scripts\Activate.ps1
Windows (Invite de commandes)	venv\Scripts\activate.bat
macOS / Linux	source venv/bin/activate

Une fois activé, le terminal affiche généralement (venv) au début de la ligne de commande. C'est le signe que tout fonctionne correctement.

À REFAIRE À CHAQUE SESSION

L'activation de l'environnement virtuel doit être répétée à chaque fois que vous ouvrez un nouveau terminal pour travailler sur ce projet. Si une commande python indique soudainement qu'un module est introuvable, vérifiez en premier lieu que l'environnement est bien activé.

Installer la bibliothèque

Une fois l'environnement virtuel activé, installez python-telegram-bot avec pip, l'outil d'installation de bibliothèques fourni avec Python :

```
$ pip install python-telegram-bot
```

L'installation télécharge et configure automatiquement tout ce qui est nécessaire.

Structurer son projet

Pour garder le projet lisible tout au long de ce guide, nous utiliserons l'organisation suivante, qui évoluera progressivement au fil des modules :

```
structure du projet
mon-bot-telegram/
├─ venv/                (environnement virtuel, généré automatiquement)
├─ bot.py               (le code principal du bot)
└─ .env                 (le token et les informations sensibles, module 14)
```

Vérifier que tout est en place

Créez un fichier `bot.py` vide dans votre dossier de projet (clic droit dans l'explorateur de VS Code > Nouveau fichier), puis exécutez cette courte vérification :

```
bot.py (test temporaire)
import telegram
print(telegram.__version__)

$ python bot.py
```

Si un numéro de version s'affiche (par exemple 22.6), l'installation est réussie et vous pouvez passer au module suivant.

06

MODULE 06

Son premier bot fonctionnel

Un bot qui répond à un message



Son premier bot fonctionnel

Ce module réunit tout ce qui a été préparé jusqu'ici pour obtenir un bot réellement fonctionnel : un programme qui répond « Bonjour ! » à tout message reçu. C'est un jalon important — une fois ce premier bot en marche, toutes les fonctionnalités suivantes viendront s'ajouter au même squelette de code.

Le code complet

Remplacez le contenu de votre fichier `bot.py` par le code suivant, en remplaçant `VOTRE_TOKEN_ICI` par le token obtenu au module 4.

```
bot.py
from telegram import Update
from telegram.ext import Application, MessageHandler, filters, ContextTypes

TOKEN = "VOTRE_TOKEN_ICI"

async def repondre(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    await update.message.reply_text("Bonjour ! Je suis en ligne.")

def main() -> None:
    application = Application.builder().token(TOKEN).build()
    application.add_handler(MessageHandler(filters.TEXT, repondre))
    application.run_polling()

if __name__ == "__main__":
    main()
```

Comprendre chaque ligne

Élément	Rôle
<code>Application.builder().token(...).build()</code>	Crée l'objet central qui représente votre bot
<code>async def repondre(...)</code>	Définit la fonction exécutée à chaque message reçu
<code>update.message.reply_text(...)</code>	Envoie une réponse dans la même conversation
<code>MessageHandler(filters.TEXT, repondre)</code>	Relie les messages texte à la fonction <code>repondre</code>
<code>application.run_polling()</code>	Démarre le bot et le fait tourner en continu

Lancer le bot

Assurez-vous que votre environnement virtuel est activé (module 5), puis exécutez :

```
$ python bot.py
```

Le programme ne renvoie rien dans le terminal — c'est normal. Il tourne silencieusement et attend des messages. Ouvrez la conversation avec votre bot dans Telegram et envoyez-lui n'importe quel message texte : il doit répondre « Bonjour ! Je suis en ligne. » en quelques instants.

COMMENT ARRÊTER LE BOT

Tant que le programme tourne, le terminal reste occupé. Pour l'arrêter, cliquez dans la fenêtre du terminal et appuyez sur Ctrl + C. Le bot cessera alors de répondre jusqu'à ce que vous relanciez la commande.

Si le bot ne répond pas

Quelques vérifications simples résolvent la grande majorité des blocages à cette étape :

- ✓ Le terminal n'affiche aucune erreur au lancement
- ✓ Le token copié est complet, sans espace avant ou après
- ✓ L'environnement virtuel est bien activé (venv) visible dans le terminal
- ✓ Vous testez bien sur la conversation avec le bon bot
- ✓ Votre connexion internet fonctionne normalement

COMPRENDRE RUN_POLLING()

Cette fonction fait tourner une boucle qui interroge Telegram en continu pour savoir si de nouveaux messages sont arrivés (voir le principe du polling au module 1). C'est la manière la plus simple de faire fonctionner un bot pendant le développement ; le module 15 présente une alternative, le webhook, plus adaptée à la production.

07

MODULE 07

Comprendre les handlers

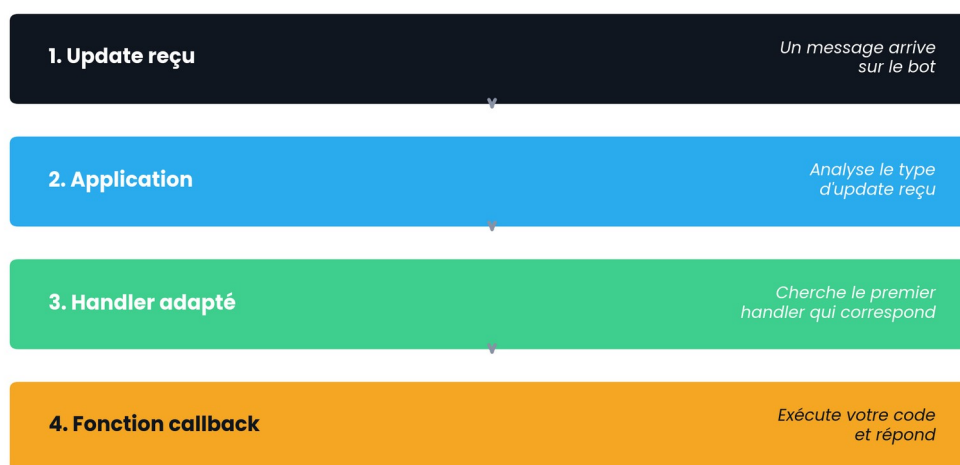
Commandes, messages et filtres



Comprendre les handlers

Un handler (gestionnaire, en français) est une fonction chargée de réagir à un type précis d'événement reçu par le bot. C'est le mécanisme central de python-telegram-bot : plutôt que d'écrire un immense bloc de code qui gère tout, on déclare une série de petits handlers, chacun responsable d'une situation particulière.

Le parcours d'un message à travers les handlers



Chaque update reçu est comparé aux handlers déclarés, dans l'ordre où ils ont été ajoutés.

Les familles de handlers les plus utilisées

Handler	Se déclenche quand...
CommandHandler	L'utilisateur envoie une commande, comme /start ou /aide
MessageHandler	L'utilisateur envoie un message (texte, photo, document...)
CallbackQueryHandler	L'utilisateur clique sur un bouton en ligne
ConversationHandler	Une série d'échanges doit suivre un ordre précis (module 10)

Le CommandHandler en détail

Une commande est un message commençant par une barre oblique, comme /start. C'est le moyen le plus explicite de déclencher une action précise dans un bot.

extrait de bot.py

```
from telegram import Update
from telegram.ext import Application, CommandHandler, ContextTypes

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text("Bienvenue ! Tapez /aide pour voir ce que je sais faire.")

def main() -> None:
    application = Application.builder().token(TOKEN).build()
    application.add_handler(CommandHandler("start", start))
    application.run_polling()
```

Le MessageHandler et les filtres

Un MessageHandler seul réagirait à tous les messages sans distinction. Les filtres permettent de restreindre son déclenchement à un type précis de contenu.

Filtre	Réagit à...
filters.TEXT	N'importe quel message texte
filters.PHOTO	Une image envoyée par l'utilisateur
filters.Document.ALL	Un fichier ou document envoyé
filters.COMMAND	Une commande (rarement utilisé directement, voir CommandHandler)
filters.TEXT & ~filters.COMMAND	Un texte qui n'est pas une commande

L'ORDRE DES HANDLERS COMPTE

Les handlers sont examinés dans l'ordre où ils ont été ajoutés avec `add_handler()`, et Telegram s'arrête généralement au premier qui correspond. Un `MessageHandler(filters.TEXT, ...)` déclaré avant un `CommandHandler` pourrait intercepter les commandes avant qu'elles n'atteignent leur handler dédié — d'où l'intérêt du filtre `~filters.COMMAND` présenté ci-dessus.

Le contexte : context

Chaque fonction de handler reçoit deux paramètres : `update`, qui contient les informations sur l'événement reçu (l'expéditeur, le texte, la conversation...), et `context`, qui donne accès à des outils utiles et à un espace de stockage temporaire propre à chaque conversation.

extrait de bot.py

```
async def afficher_id(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    utilisateur = update.effective_user
    await update.message.reply_text(
        f"Bonjour {utilisateur.first_name}, votre identifiant est
{utilisateur.id}"
    )
```

08

MODULE 08

Créer des commandes personnalisées

Construire une vraie interface conversationnelle



Créer des commandes personnalisées

Avec les handlers en main, il devient possible de construire une véritable interface conversationnelle : plusieurs commandes, chacune avec un rôle précis, qui donnent à l'utilisateur le sentiment de dialoguer avec un assistant cohérent plutôt qu'un simple script.

Un bot à plusieurs commandes

Voici une version enrichie du bot, avec trois commandes distinctes : /start pour l'accueil, /aide pour la liste des commandes, et /heure pour afficher l'heure actuelle.

```
bot.py

from datetime import datetime
from telegram import Update
from telegram.ext import Application, CommandHandler, ContextTypes

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text(
        "Bienvenue ! Voici ce que je sais faire :\n"
        "/aide - afficher les commandes disponibles\n"
        "/heure - afficher l'heure actuelle"
    )

async def aide(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text(
        "Commandes disponibles :\n/start\n/aide\n/heure"
    )

async def heure(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    maintenant = datetime.now().strftime("%H:%M")
    await update.message.reply_text(f"Il est actuellement {maintenant}.")

def main() -> None:
    application = Application.builder().token(TOKEN).build()
    application.add_handler(CommandHandler("start", start))
    application.add_handler(CommandHandler("aide", aide))
    application.add_handler(CommandHandler("heure", heure))
    application.run_polling()

if __name__ == "__main__":
    main()
```

Récupérer des arguments dans une commande

Une commande peut aussi recevoir des mots supplémentaires, appelés arguments. Ils sont accessibles via `context.args`, sous la forme d'une liste de mots.

extrait de `bot.py`

```
async def saluer(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    if context.args:
        prenom = context.args[0]
        await update.message.reply_text(f"Bonjour {prenom} !")
    else:
        await update.message.reply_text("Utilisation : /saluer VotrePrenom")
```

Envoyer `/saluer Alex` déclenche cette fonction avec `context.args` égal à `["Alex"]`, et le bot répond « Bonjour Alex ! ».

Déclarer les commandes auprès de BotFather

Pour que vos commandes apparaissent dans le menu (l'icône « / » à côté du champ de saisie sur Telegram), déclarez-les auprès de BotFather avec la commande `/setcommands`, en respectant ce format :

message à envoyer à BotFather

```
start - Démarrer le bot
aide - Afficher les commandes disponibles
heure - Afficher l'heure actuelle
```

BONNE PRATIQUE

Gardez chaque fonction de commande courte et responsable d'une seule chose. Si une commande devient complexe, il est presque toujours préférable de découper sa logique en plusieurs petites fonctions auxiliaires appelées depuis le handler, plutôt que d'accumuler du code dans une seule fonction.

Répondre différemment selon le contexte

Un bot bien conçu adapte parfois sa réponse selon qui écrit ou dans quel type de conversation (message privé, groupe). Ces informations sont disponibles via `update.effective_chat` et `update.effective_user`.

extrait de `bot.py`

```
async def info_conversation(update: Update, context:
ContextTypes.DEFAULT_TYPE) -> None:
    type_conversation = update.effective_chat.type # "private", "group",
"supergroup"
    if type_conversation == "private":
        await update.message.reply_text("Nous discutons en privé.")
    else:
        await update.message.reply_text("Nous sommes dans un groupe.")
```

09

MODULE 09

Boutons et menus interactifs

Claviers en ligne et claviers personnalisés



Boutons et menus interactifs

Taper des commandes fonctionne bien, mais devient vite fastidieux pour l'utilisateur. Telegram propose deux systèmes de boutons qui rendent un bot beaucoup plus agréable à utiliser : le clavier en ligne (inline keyboard), attaché à un message précis, et le clavier personnalisé (reply keyboard), qui remplace le clavier habituel de l'utilisateur.

Le clavier en ligne (InlineKeyboardMarkup)

Ces boutons apparaissent directement sous un message et déclenchent un événement particulier, appelé callback query, lorsqu'on clique dessus.

```
extrait de bot.py

from telegram import InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import CallbackQueryHandler

async def menu(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    clavier = [
        [InlineKeyboardButton("Voir mon profil", callback_data="profil")],
        [InlineKeyboardButton("Paramètres", callback_data="parametres")],
    ]
    await update.message.reply_text(
        "Que souhaitez-vous faire ?",
        reply_markup=InlineKeyboardMarkup(clavier)
    )

async def gerer_clic(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    query = update.callback_query
    await query.answer() # accuse réception, sinon le bouton reste "en
chargement"
    if query.data == "profil":
        await query.edit_message_text("Voici votre profil.")
    elif query.data == "parametres":
        await query.edit_message_text("Voici les paramètres.")
```

Le handler correspondant s'enregistre ainsi :

```
extrait de main()

application.add_handler(CommandHandler("menu", menu))
application.add_handler(CallbackQueryHandler(gerer_clic))
```


NE JAMAIS OUBLIER `QUERY.ANSWER()`

Sans cet appel, le bouton cliqué par l'utilisateur reste visuellement « en chargement » pendant plusieurs secondes avant d'expirer. Appelez-le systématiquement au tout début de votre fonction, même si vous ne l'utilisez pas pour afficher un message.

Distinguer plusieurs boutons avec `callback_data`

Le champ `callback_data` est une chaîne de texte que vous choisissez librement (limitée à 64 caractères) : c'est elle qui permet à `gerer_clic` de savoir quel bouton précis a été pressé. Une convention courante consiste à y encoder plusieurs informations séparées par un caractère, comme "produit:12" pour identifier un article précis.

Le clavier personnalisé (`ReplyKeyboardMarkup`)

Contrairement au clavier en ligne, ce type de clavier remplace le clavier habituel de l'utilisateur par des boutons prédéfinis. Il convient bien aux menus principaux ou aux réponses à choix limité.

```
extrait de bot.py
from telegram import ReplyKeyboardMarkup

async def afficher_menu_principal(update: Update, context:
ContextTypes.DEFAULT_TYPE) -> None:
    clavier = [["Commander", "Suivre ma commande"], ["Contacter le support"]]
    await update.message.reply_text(
        "Choisissez une option :",
        reply_markup=ReplyKeyboardMarkup(clavier, resize_keyboard=True)
    )
```

Un clic sur l'un de ces boutons envoie simplement le texte affiché comme un message normal : il doit donc être intercepté par un `MessageHandler` classique, comme n'importe quel texte tapé manuellement par l'utilisateur.

Quand utiliser lequel ?

Situation	Clavier recommandé
Options liées à un message précis (voter, choisir un article)	<code>InlineKeyboardMarkup</code>
Navigation principale, disponible en permanence	<code>ReplyKeyboardMarkup</code>
Réponse qui doit rester consultable après avoir été cliquée	<code>InlineKeyboardMarkup</code>
Simplifier la saisie pour des réponses courtes et répétées	<code>ReplyKeyboardMarkup</code>

10

MODULE 10

Gérer une conversation à états

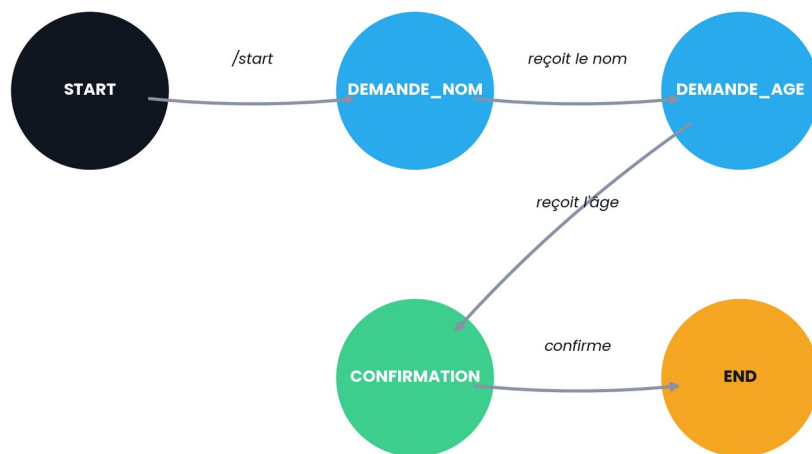
Le ConversationHandler en détail



Gérer une conversation à états

Certaines interactions nécessitent plusieurs échanges dans un ordre précis : un formulaire d'inscription, un quiz, une commande guidée. Le ConversationHandler permet de structurer ce type d'échange comme une suite d'étapes, appelées états, où chaque état attend un type précis de réponse avant de passer au suivant.

Exemple de machine à états : un formulaire en 3 étapes



Un exemple simple : demander un nom, puis un âge, avant de confirmer.

Construire un formulaire en trois étapes

Reprenons l'exemple illustré ci-dessus : un bot qui demande le nom puis l'âge de l'utilisateur avant de confirmer les informations reçues.

```

extrait de bot.py

from telegram import Update
from telegram.ext import (
    Application, CommandHandler, MessageHandler, ConversationHandler,
    filters, ContextTypes
)

NOM, AGE = range(2)

async def demarrer_formulaire(update: Update, context:
ContextTypes.DEFAULT_TYPE) -> int:
    await update.message.reply_text("Quel est votre prénom ?")
    return NOM
  
```

```

async def recevoir_nom(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
int:
    context.user_data["nom"] = update.message.text
    await update.message.reply_text("Quel est votre âge ?")
    return AGE

async def recevoir_age(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
int:
    context.user_data["age"] = update.message.text
    nom = context.user_data["nom"]
    age = context.user_data["age"]
    await update.message.reply_text(f"Merci {nom}, {age} ans bien noté !")
    return ConversationHandler.END

async def annuler(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    await update.message.reply_text("Formulaire annulé.")
    return ConversationHandler.END

```

Assembler le ConversationHandler

```

extrait de main()

formulaire = ConversationHandler(
    entry_points=[CommandHandler("inscription", demarrer_formulaire)],
    states={
        NOM: [MessageHandler(filters.TEXT & ~filters.COMMAND, recevoir_nom)],
        AGE: [MessageHandler(filters.TEXT & ~filters.COMMAND, recevoir_age)],
    },
    fallbacks=[CommandHandler("annuler", annuler)],
)

application.add_handler(formulaire)

```

Comprendre les trois parties

Paramètre	Rôle
entry_points	La ou les commandes qui déclenchent le début de la conversation
states	Un dictionnaire associant chaque état à la liste de handlers qui peuvent s'y déclencher
fallbacks	Les handlers actifs à tout moment, comme une commande d'annulation

À chaque étape, la fonction retourne la constante de l'état suivant (NOM, AGE) pour indiquer où poursuivre, ou ConversationHandler.END pour signaler que la conversation est terminée et que le bot doit revenir à son comportement normal.

TOUJOURS PRÉVOIR UNE SORTIE

Un utilisateur peut se tromper, changer d'avis ou taper une commande imprévue en pleine conversation. Prévoir un fallback comme `/annuler` évite qu'il ne reste bloqué dans un état sans pouvoir en sortir.

Où sont stockées les réponses ?

Le dictionnaire `context.user_data` permet de conserver des informations tout au long de la conversation d'un utilisateur donné. Il est automatiquement réinitialisé lorsque la conversation se termine (ou peut être vidé manuellement avec `context.user_data.clear()` si besoin). Pour conserver des données au-delà d'une seule session, direction le module suivant.

11

MODULE 11

Stocker des données

Fichiers JSON et base de données SQLite

Stocker des données

`context.user_data`, vu au module précédent, ne survit pas au redémarrage du bot : dès que le programme s'arrête, tout est perdu. Pour retenir durablement des informations — les préférences d'un utilisateur, une liste d'éléments, un historique — il faut les enregistrer quelque part de façon permanente.

Solution simple : un fichier JSON

JSON est un format texte qui permet de représenter des données structurées (listes, dictionnaires) de façon lisible. C'est une solution suffisante pour un bot avec peu d'utilisateurs ou des besoins simples.

```
stockage.py

import json
import os

FICHIER_DONNEES = "donnees.json"

def charger_donnees():
    if not os.path.exists(FICHIER_DONNEES):
        return {}
    with open(FICHIER_DONNEES, "r", encoding="utf-8") as f:
        return json.load(f)

def sauvegarder_donnees(donnees):
    with open(FICHIER_DONNEES, "w", encoding="utf-8") as f:
        json.dump(donnees, f, ensure_ascii=False, indent=2)
```

Ces deux fonctions peuvent ensuite être utilisées dans vos handlers pour lire et modifier les informations d'un utilisateur, identifié par son identifiant Telegram :

```
extrait de bot.py

async def enregistrer_prenom(update: Update, context:
ContextTypes.DEFAULT_TYPE) -> None:
    donnees = charger_donnees()
    id_utilisateur = str(update.effective_user.id)
    donnees[id_utilisateur] = {"prenom": context.args[0]}
    sauvegarder_donnees(donnees)
    await update.message.reply_text("Prénom enregistré !")
```

LIMITE DU FICHIER JSON

Cette solution devient fragile avec beaucoup d'utilisateurs simultanés : deux écritures en même temps peuvent se marcher dessus et corrompre le fichier. Elle convient très bien pour apprendre et pour de petits projets, mais une base de données est préférable au-delà de quelques dizaines d'utilisateurs actifs.

Solution plus robuste : SQLite

SQLite est une base de données légère qui ne nécessite aucune installation ni serveur séparé : elle fonctionne directement à partir d'un simple fichier, et Python sait la manipuler nativement grâce au module `sqlite3`, déjà inclus.

```
base_donnees.py (à exécuter une fois)
import sqlite3

connexion = sqlite3.connect("bot.db")
curseur = connexion.cursor()

curseur.execute("""
    CREATE TABLE IF NOT EXISTS utilisateurs (
        id INTEGER PRIMARY KEY,
        prenom TEXT
    )
""")
connexion.commit()
```

Une fois la table créée, l'ajout et la lecture de données se font avec des requêtes SQL, un langage dédié à l'interrogation de bases de données :

```
base_donnees.py
def enregistrer_utilisateur(id_utilisateur, prenom):
    curseur.execute(
        "INSERT OR REPLACE INTO utilisateurs (id, prenom) VALUES (?, ?)",
        (id_utilisateur, prenom)
    )
    connexion.commit()

def recuperer_prenom(id_utilisateur):
    curseur.execute("SELECT prenom FROM utilisateurs WHERE id = ?",
        (id_utilisateur,))
    resultat = curseur.fetchone()
    return resultat[0] if resultat else None
```

POURQUOI LE POINT D'INTERROGATION ?

Le symbole `?` dans la requête SQL, associé au tuple de valeurs passé en second argument, protège contre l'injection SQL — une technique malveillante qui consiste à glisser du code dans une valeur censée être une simple donnée. Ne construisez jamais une requête en collant directement du texte utilisateur dans la chaîne SQL.

Quelle solution choisir ?

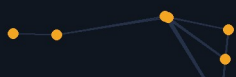
Besoin	Solution recommandée
Prototype rapide, peu d'utilisateurs	Fichier JSON
Bot en usage réel, plusieurs utilisateurs	SQLite
Application à grande échelle, plusieurs serveurs	Base de données externe (PostgreSQL, MySQL)

12

MODULE 12

Envoyer et recevoir des médias

Photos, documents et fichiers



Envoyer et recevoir des médias

Un bot n'est pas limité au texte : il peut envoyer et recevoir des photos, des documents, des fichiers audio ou des vidéos. Ce module couvre les cas les plus courants.

Envoyer une image

Une image peut être envoyée depuis un fichier local ou depuis une URL publique.

```
extrait de bot.py

async def envoyer_photo(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    with open("images/logo.png", "rb") as fichier:
        await update.message.reply_photo(photo=fichier, caption="Voici notre
logo !")
```

Envoyer un document

```
extrait de bot.py

async def envoyer_document(update: Update, context: ContextTypes.DEFAULT_TYPE)
-> None:
    with open("fichiers/guide.pdf", "rb") as fichier:
        await update.message.reply_document(document=fichier,
filename="guide.pdf")
```

Recevoir une image envoyée par l'utilisateur

Lorsqu'un utilisateur envoie une photo, Telegram la transmet en plusieurs résolutions. `update.message.photo[-1]` correspond à la version en plus haute résolution disponible.

```
extrait de bot.py

from telegram.ext import MessageHandler, filters

async def recevoir_photo(update: Update, context: ContextTypes.DEFAULT_TYPE) -
> None:
    fichier_photo = await update.message.photo[-1].get_file()
    await fichier_photo.download_to_drive("images/recue.jpg")
    await update.message.reply_text("Photo bien reçue, merci !")

application.add_handler(MessageHandler(filters.PHOTO, recevoir_photo))
```

Recevoir un document

extrait de bot.py

```
async def recevoir_document(update: Update, context:
ContextTypes.DEFAULT_TYPE) -> None:
    document = update.message.document
    fichier = await document.get_file()
    await fichier.download_to_drive(f"fichiers/{document.file_name}")
    await update.message.reply_text(f"Fichier « {document.file_name} »
enregistré.")

application.add_handler(MessageHandler(filters.Document.ALL,
recevoir_document))
```

ATTENTION À LA TAILLE DES FICHIERS

L'API Telegram limite le téléchargement des fichiers par un bot à 20 Mo (l'envoi peut aller jusqu'à 50 Mo dans certains cas). Pour des fichiers plus volumineux, il faut recourir à l'API Telegram locale (Bot API Server auto-hébergé), une configuration avancée qui dépasse le cadre de ce guide.

Récapitulatif des méthodes d'envoi

Type de contenu	Méthode d'envoi
Photo	reply_photo()
Document / fichier	reply_document()
Audio	reply_audio()
Vidéo	reply_video()
Message vocal	reply_voice()
Localisation	reply_location()

Toutes ces méthodes acceptent soit un fichier ouvert localement (comme dans les exemples ci-dessus), soit une URL publique, soit un file_id déjà connu de Telegram — une astuce utile pour renvoyer rapidement un contenu déjà envoyé une première fois, sans le retélécharger.

13

MODULE 13

Gérer les erreurs et journaliser

Rendre son bot robuste et observable



Gérer les erreurs et journaliser

Un bot destiné à un usage réel doit survivre à l'imprévu : un utilisateur qui envoie un format inattendu, une connexion internet qui coupe un instant, un service externe qui répond mal. Sans précaution, une seule erreur non gérée peut arrêter le programme entier. Ce module montre comment rendre un bot robuste et comment garder une trace de ce qui se passe.

Journaliser avec le module logging

Plutôt que d'utiliser `print()` pour surveiller ce que fait votre bot, le module `logging`, inclus avec Python, permet d'enregistrer des messages classés par niveau d'importance, avec horodatage automatique.

```
extrait de bot.py
import logging

logging.basicConfig(
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    level=logging.INFO
)
logger = logging.getLogger(__name__)

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    logger.info(f"Commande /start reçue de {update.effective_user.id}")
    await update.message.reply_text("Bienvenue !")
```

Niveau	Utilisation typique
DEBUG	Détails techniques utiles seulement pendant le développement
INFO	Déroulement normal du programme (une commande reçue, une action effectuée)
WARNING	Une situation inhabituelle mais non bloquante
ERROR	Une erreur qui a empêché une action de se terminer correctement

Un gestionnaire d'erreurs global

`python-telegram-bot` permet de déclarer une fonction spéciale, appelée automatiquement chaque fois qu'une erreur survient dans n'importe quel handler, sans faire planter le programme.

```

extrait de main()

async def gerer_erreur(update: object, context: ContextTypes.DEFAULT_TYPE) ->
None:
    logger.error(f"Une erreur est survenue : {context.error}")
    if isinstance(update, Update) and update.effective_message:
        await update.effective_message.reply_text(
            "Une erreur inattendue est survenue. Réessayez dans un instant."
        )

application.add_error_handler(gerer_erreur)

```

POURQUOI C'EST ESSENTIEL

Sans ce gestionnaire, une erreur dans un seul handler (par exemple une division par zéro, ou un fichier introuvable) reste silencieuse dans les versions récentes de la bibliothèque : le bot continue de tourner, mais l'utilisateur concerné ne reçoit jamais de réponse et vous n'êtes prévenu de rien sans consulter les journaux.

Se protéger d'une entrée inattendue

Au-delà du gestionnaire global, il reste utile d'anticiper les cas prévisibles directement dans vos handlers, avec des blocs try / except, pour offrir un message d'erreur clair plutôt qu'un échec générique.

```

extrait de bot.py

async def diviser(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    try:
        a, b = context.args[0], context.args[1]
        resultat = float(a) / float(b)
        await update.message.reply_text(f"Résultat : {resultat}")
    except IndexError:
        await update.message.reply_text("Utilisation : /diviser 10 2")
    except ValueError:
        await update.message.reply_text("Merci d'indiquer deux nombres valides.")
    except ZeroDivisionError:
        await update.message.reply_text("Impossible de diviser par zéro.")

```

Où consulter les journaux en production

Une fois le bot hébergé sur un serveur (module 15), les messages enregistrés par logging apparaissent généralement dans les journaux fournis par la plateforme d'hébergement (logs Railway, journal systemd, fichier de sortie selon la méthode utilisée). Consulter ces journaux régulièrement, surtout dans les premiers jours suivant une mise en ligne, permet de repérer rapidement les problèmes réels rencontrés par vos utilisateurs.

14

MODULE 14

Sécuriser son bot

Variables d'environnement et bonnes pratiques



Sécuriser son bot

Jusqu'ici, le token a probablement été écrit directement dans le fichier `bot.py`, ce qui fonctionne mais reste risqué : ce fichier finit souvent par être partagé, sauvegardé en ligne ou publié sans que l'on y pense. Ce module montre la bonne pratique standard pour séparer les informations sensibles du code lui-même.

Utiliser un fichier `.env`

Un fichier `.env` contient des variables sous forme clé=valeur, chargées par le programme au démarrage, sans jamais apparaître dans le code source lui-même.

Installez d'abord la bibliothèque qui permet de lire ce type de fichier :

```
$ pip install python-dotenv
```

Créez ensuite un fichier `.env` à la racine de votre projet :

```
.env  
TELEGRAM_BOT_TOKEN=votre_token_ici
```

Et chargez-le au tout début de votre `bot.py` :

```
extrait de bot.py  
import os  
from dotenv import load_dotenv  
  
load_dotenv()  
TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
```

NE JAMAIS PUBLIER `.ENV`

Si votre projet est suivi avec Git ou publié sur GitHub, ajoutez immédiatement un fichier `.gitignore` contenant la ligne `.env`, pour être certain que ce fichier ne soit jamais envoyé publiquement, même par erreur.

```
.gitignore  
venv/  
.env  
__pycache__/  
*.pyc
```

Que faire si un token a fuité ?

Si vous pensez que votre token a été exposé (publié par erreur, partagé, visible dans une capture d'écran), régénérez-le immédiatement : tout ancien token cesse alors de fonctionner.

1. Ouvrez la conversation avec BotFather.
2. Envoyez `/mybots`, puis sélectionnez votre bot.
3. Choisissez Bot Settings, puis Revoke current token.
4. Copiez le nouveau token et mettez à jour votre fichier `.env`.

Limiter qui peut utiliser certaines commandes

Certaines commandes (administration, statistiques, actions sensibles) ne doivent être accessibles qu'à vous-même ou à un groupe restreint d'utilisateurs. On peut restreindre l'accès en comparant l'identifiant de l'utilisateur à une liste autorisée.

```
extrait de bot.py
ADMINISTRATEURS = [123456789] # remplacez par votre identifiant Telegram

async def commande_admin(update: Update, context: ContextTypes.DEFAULT_TYPE) -
> None:
    if update.effective_user.id not in ADMINISTRATEURS:
        await update.message.reply_text("Cette commande est réservée aux
administrateurs.")
        return
    await update.message.reply_text("Commande administrateur exécutée.")
```

Pour connaître votre propre identifiant Telegram, envoyez n'importe quel message à votre bot et affichez `update.effective_user.id` dans une commande temporaire, ou utilisez un bot dédié comme `@userinfobot`.

Checklist de sécurité avant mise en ligne

- ✓ Le token est stocké dans `.env`, jamais écrit en dur dans le code
- ✓ `.env` figure dans `.gitignore` si le projet est versionné avec Git
- ✓ Les commandes sensibles vérifient l'identité de l'utilisateur
- ✓ Les entrées utilisateur sont validées avant d'être utilisées (module 13)
- ✓ Aucune information personnelle n'est journalisée inutilement dans les logs

15

MODULE 15

Héberger et déployer son bot

Faire tourner son bot 24h/24

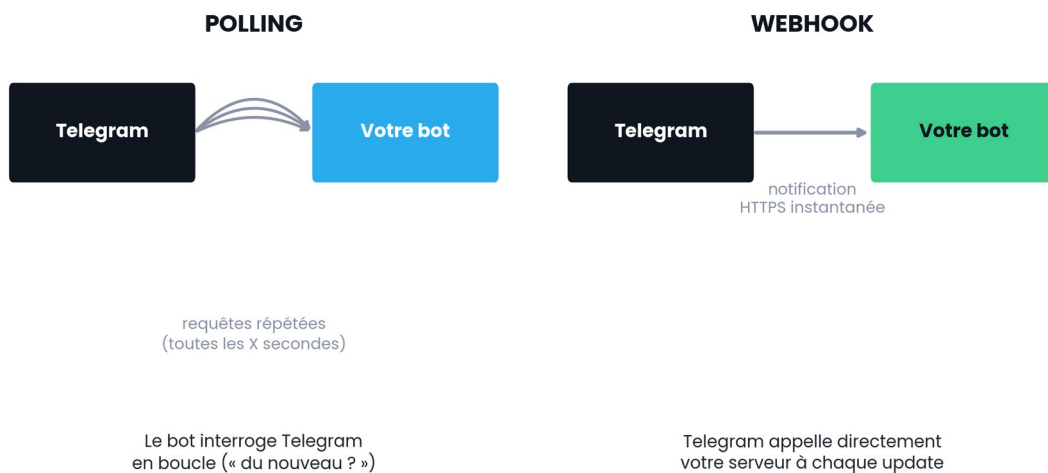


Héberger et déployer son bot

Tant que le bot ne tourne que sur votre ordinateur, il s'arrête dès que vous fermez le terminal, éteignez votre machine ou perdez la connexion internet. Pour qu'il reste disponible en continu, il faut le faire tourner sur un serveur : une machine allumée en permanence, quelque part sur internet.

Polling ou webhook ?

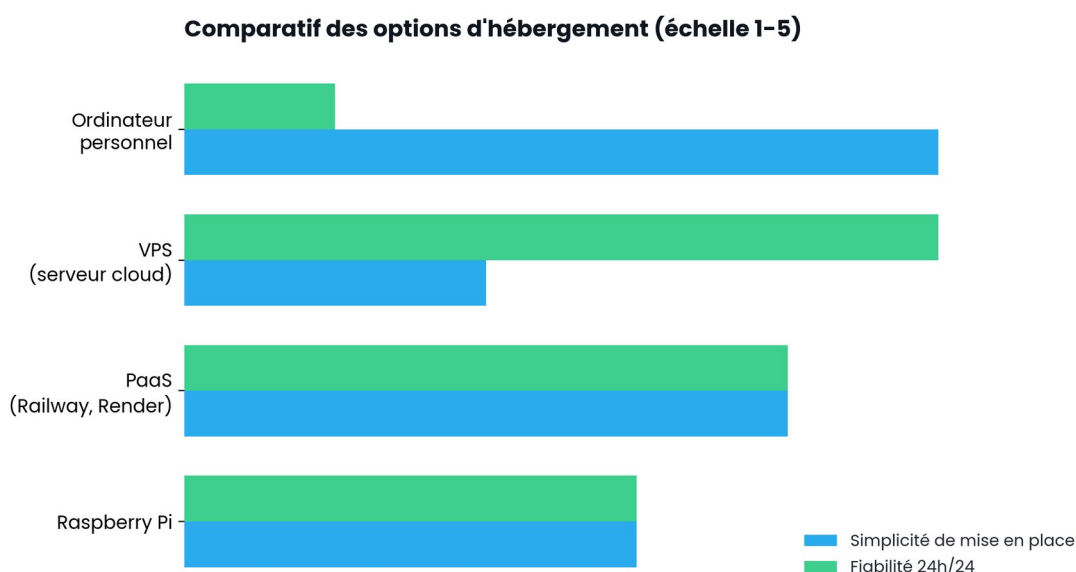
Deux méthodes permettent à votre bot de recevoir les messages une fois hébergé. Le module 1 a introduit ces notions ; voici la comparaison pratique pour choisir.



Le polling reste la méthode la plus simple à mettre en œuvre pour un bot hébergé sur un petit serveur.

Pour la grande majorité des projets personnels ou de petite taille, `run_polling()` (déjà utilisé depuis le module 6) suffit amplement une fois déployé sur un serveur : il n'est pas nécessaire de passer à un webhook pour commencer. Le webhook devient intéressant pour des bots à fort trafic, où l'attente du polling engendrerait un délai de réponse perceptible.

Comparer les options d'hébergement



Pour un premier déploiement, une plateforme PaaS offre le meilleur compromis simplicité / fiabilité.

Option	Idéal pour
PaaS (Railway, Render, Fly.io)	Démarrer rapidement sans gérer de serveur, souvent avec un niveau gratuit
VPS (serveur privé virtuel)	Un contrôle total, pour un coût mensuel modeste, avec un peu de configuration
Raspberry Pi à la maison	Un projet personnel, sans coût récurrent, mais dépendant de votre connexion

Déployer sur une plateforme PaaS (exemple générique)

Les plateformes de ce type suivent généralement la même logique : connecter votre code (souvent via un dépôt Git), définir la commande de lancement, et fournir les variables d'environnement (dont le token).

1. **Créez un fichier `requirements.txt`** qui liste les bibliothèques utilisées par votre projet.
2. **Créez un compte** sur la plateforme choisie et reliez votre projet (dépôt Git ou envoi direct des fichiers).
3. **Renseignez la variable d'environnement** `TELEGRAM_BOT_TOKEN` dans les réglages de la plateforme, avec la valeur de votre token.
4. **Définissez la commande de démarrage** : `python bot.py`
5. **Lancez le déploiement** et consultez les journaux pour vérifier que le bot démarre sans erreur.

```
requirements.txt  
python-telegram-bot  
python-dotenv
```

GÉNÉRER REQUIREMENTS.TXT AUTOMATIQUEMENT

Si vous avez installé vos bibliothèques dans un environnement virtuel actif, la commande `pip freeze > requirements.txt` génère ce fichier automatiquement à partir de ce qui est réellement installé.

Déployer sur un VPS (principe général)

Un VPS est une machine Linux distante à laquelle on se connecte à distance. Une fois le code transféré et les dépendances installées comme sur votre ordinateur, il faut garder le programme actif même après avoir fermé la connexion — un gestionnaire de processus comme `systemd` (sur Linux) permet de démarrer le bot automatiquement et de le relancer en cas de plantage.

```
/etc/systemd/system/mon-bot.service  
  
[Unit]  
Description=Mon bot Telegram  
After=network.target  
  
[Service]  
WorkingDirectory=/home/utilisateur/mon-bot-telegram  
ExecStart=/home/utilisateur/mon-bot-telegram/venv/bin/python bot.py  
Restart=always  
  
[Install]  
WantedBy=multi-user.target  
  
$ sudo systemctl enable mon-bot  
$ sudo systemctl start mon-bot
```

Ces commandes activent le service au démarrage du serveur et le lancent immédiatement.

Checklist avant la mise en production

- ✓ Le token est fourni via une variable d'environnement, jamais écrit en dur
- ✓ Un gestionnaire d'erreurs global est en place (module 13)
- ✓ Le programme redémarre automatiquement en cas de plantage
- ✓ Les journaux sont consultables facilement depuis la plateforme choisie
- ✓ Le bot a été testé une dernière fois après le déploiement, en conditions réelles

16

MODULE 16

Aller plus loin

Tâches planifiées, API externes et IA



Aller plus loin

Une fois les bases solidement en place, plusieurs directions permettent d'enrichir un bot. Ce module donne un aperçu de trois d'entre elles, suffisant pour démarrer une exploration autonome.

Planifier des tâches avec JobQueue

JobQueue permet d'exécuter une action à un moment précis ou à intervalles réguliers, sans attendre un message de l'utilisateur — utile pour des rappels, des résumés quotidiens ou des vérifications périodiques.

```
extrait de bot.py

async def rappel_quotidien(context: ContextTypes.DEFAULT_TYPE) -> None:
    await context.bot.send_message(chat_id=123456789, text="N'oubliez pas
    votre point du jour !")

def main() -> None:
    application = Application.builder().token(TOKEN).build()
    application.job_queue.run_daily(rappel_quotidien,
    time=datetime.time(hour=9, minute=0))
    application.run_polling()
```

Appeler une API externe

Un bot devient bien plus utile lorsqu'il peut aller chercher une information à l'extérieur : météo, cours de bourse, actualités, ou tout autre service exposant une API. La bibliothèque `httpx` (ou `requests`) permet d'effectuer ces appels facilement.

```
$ pip install httpx
```

```
extrait de bot.py
```

```
import httpx
```

```
async def meteo(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    async with httpx.AsyncClient() as client:
        reponse = await client.get("https://api.exemple-meteo.com/paris")
        donnees = reponse.json()
        await update.message.reply_text(f"Il fait {donnees['temperature']}°C à
    Paris.")
```


TOUJOURS UTILISER UN CLIENT ASYNCHRONE

Dans un bot construit avec `python-telegram-bot`, un appel réseau classique (comme `requests.get`) bloquerait tout le programme pendant l'attente de la réponse. `httpx.AsyncClient`, utilisé avec `await`, laisse le bot continuer à traiter d'autres messages pendant l'attente.

Connecter une intelligence artificielle

De nombreux bots utilisent aujourd'hui un modèle de langage pour générer des réponses plus naturelles ou traiter des requêtes complexes. Le principe reste similaire à l'appel d'une API externe : envoyer le message de l'utilisateur au service d'IA, puis renvoyer sa réponse.

```
extrait de bot.py
import anthropic

client_ia = anthropic.Anthropic() # la clé API se configure via une variable
d'environnement

async def discuter(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    reponse = client_ia.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=500,
        messages=[{"role": "user", "content": update.message.text}]
    )
    await update.message.reply_text(reponse.content[0].text)
```

Gérer un bot dans un groupe

Ajouté à un groupe, un bot ne reçoit par défaut que les messages qui le mentionnent explicitement, sauf si le mode « confidentialité » est désactivé auprès de BotFather (`/setprivacy`). Quelques permissions supplémentaires (épingler un message, supprimer un message, gérer les membres) doivent être accordées manuellement au bot par un administrateur du groupe.

Pour continuer à apprendre

- La documentation officielle de `python-telegram-bot` regroupe une référence complète et des exemples couvrant des cas avancés
- La documentation de l'API Telegram Bot détaille chaque méthode et chaque type d'objet disponible
- Lire le code d'autres bots open source reste l'un des moyens les plus rapides de découvrir de nouvelles pratiques

17

MODULE 17

Votre feuille de route

Passer de la théorie à un bot en production



Votre feuille de route

Ce dernier module propose un plan concret sur douze semaines pour transformer les connaissances de ce guide en un bot réellement en ligne, en avançant par étapes réalistes plutôt qu'en essayant de tout construire d'un coup.

Feuille de route d'apprentissage — 12 semaines



Une progression pensée pour rester motivante : des résultats visibles dès les deux premières semaines.

Semaines 1 à 2 — Les bases

- ✓ Installer Python et VS Code, vérifier que tout fonctionne
- ✓ Créer un bot avec BotFather et récupérer son token
- ✓ Installer python-telegram-bot dans un environnement virtuel
- ✓ Faire fonctionner le premier bot qui répond à un message
- ✓ Ajouter deux ou trois commandes personnalisées simples

Semaines 3 à 6 — Les fonctionnalités

- ✓ Ajouter des boutons en ligne pour au moins une interaction
- ✓ Construire une conversation guidée avec ConversationHandler
- ✓ Mettre en place un stockage de données (JSON ou SQLite)
- ✓ Permettre l'envoi ou la réception d'au moins un type de média
- ✓ Tester le bot avec des proches pour recueillir un premier retour

Semaines 7 à 12 — La mise en production

- ✓ Ajouter un gestionnaire d'erreurs global et de la journalisation

- ✓ Déplacer le token vers un fichier .env, sécuriser les commandes sensibles
- ✓ Choisir une solution d'hébergement et déployer le bot
- ✓ Observer les journaux pendant une semaine pour repérer les problèmes réels
- ✓ Ajouter une fonctionnalité supplémentaire choisie au module 16

UN PROJET N'EST JAMAIS VRAIMENT TERMINÉ

Un bot en usage réel continue d'évoluer : nouvelles fonctionnalités, corrections, ajustements suite aux retours des utilisateurs. Considérez la fin de cette feuille de route comme un point de départ solide plutôt que comme une ligne d'arrivée.

Idées de projets pour continuer à pratiquer

Projet	Notions mobilisées
Bot de liste de tâches personnelle	Stockage, commandes, boutons
Bot de quiz avec score	ConversationHandler, stockage, boutons
Bot de veille (alertes automatiques)	JobQueue, appel d'API externe
Bot d'assistant conversationnel	API d'intelligence artificielle, historique de conversation
Bot de modération pour un groupe	Permissions, filtres avancés, administration

CONCLUSION

Ce qu'il faut retenir

Créer un bot Telegram ne demande pas de compétences exceptionnelles : il s'agit d'assembler, méthodiquement, un petit nombre de briques — handlers, états, stockage, sécurité, hébergement — chacune simple prise isolément. La complexité d'un bot vient rarement d'une brique en particulier, mais de la façon dont elles s'articulent les unes aux autres à mesure que le projet grandit.

Si vous deviez ne retenir que quelques principes de ce guide, ce seraient les suivants :

- Avancer par petites étapes testables plutôt que d'écrire beaucoup de code avant le premier test
- Garder chaque handler simple et responsable d'une seule chose
- Séparer systématiquement les informations sensibles du code source
- Prévoir la gestion des erreurs dès le début plutôt que de l'ajouter en urgence plus tard
- Ne pas viser la perfection avant la mise en ligne : un bot en production s'améliore avec l'usage réel

POUR CONTINUER À PROGRESSER

Reprenez votre feuille de route du module 17 dans trois mois, comparez votre bot à vos ambitions de départ, et laissez-le évoluer. La pratique sur un projet réel enseigne toujours plus vite que la lecture seule.

Lexique des termes essentiels

Terme	Définition
Token	Clé secrète qui identifie et permet de contrôler un bot
API	Interface qui permet à un programme de communiquer avec un service (ici, Telegram)
Update	Un événement reçu par le bot (message, clic sur un bouton, etc.)
Handler	Fonction chargée de réagir à un type précis d'update
Polling	Le bot interroge Telegram en boucle pour détecter les nouveaux messages
Webhook	Telegram notifie directement le serveur du bot à chaque update
Environnement virtuel	Espace Python isolé propre à un projet
async / await	Mécanisme Python permettant d'attendre une opération sans bloquer le programme
ConversationHandler	Handler qui structure un échange en plusieurs étapes ordonnées
Callback query	Événement déclenché par le clic sur un bouton en ligne
JSON	Format texte structuré utilisé pour stocker ou échanger des données
SQLite	Base de données légère fonctionnant à partir d'un simple fichier
VPS	Serveur privé virtuel, une machine distante louée pour héberger une application
PaaS	Plateforme d'hébergement clé en main (Platform as a Service)

MERCI D'AVOIR SUIVI CE GUIDE

Création d'un Bot Telegram — Guide Complet

Le H